

1 Roteiro Prático 04 — Arquitetura Frontend Modular: ES6 Modules & Padrão em Camadas

Objetivos

Objetivos da Atividade (Módulo 2 — ES6 Modules & Padrão em Camadas)

Ao final desta prática orientada em laboratório, o estudante será capaz de:

- Superar a arquitetura monolítica de arquivo único (`app.js`), organizando o frontend em múltiplos módulos especializados utilizando **ES6 Modules** (`import` e `export`);
- Declarar o ponto de entrada modular na página web através da diretiva `<script type="module" src="js/main.js"></script>`;
- Compreender e aplicar a separação estrita de responsabilidades no cliente em três camadas:
 - **Camada de Serviços (`services/`):** Regras de negócio, cálculos e persistência pura no `localStorage` sem acoplamento ao DOM;
 - **Camada de Visão (`views/`):** Renderização de cards, tabelas, modais e Toasts via *template literals* do Bootstrap 5;
 - **Camada Utilitária (`utils/`):** Funções puras reutilizáveis de formatação (moeda brasileira, telefone, máscaras);
 - **Orquestrador Central (`main.js`):** Captura de eventos do usuário e coordenação entre camadas.
- Implementar **validação visual no cliente** com feedback imediato ao usuário utilizando as classes nativas do Bootstrap 5 (`.is-valid`, `.is-invalid` e `.invalid-feedback`);
- Disparar notificações flutuantes dinâmicas (**Bootstrap Toasts**) para confirmar operações de cadastro e exclusão;
- Implementar filtragem reativa em tempo real por categoria de serviço na vitrine comunitária.

1.1 Introdução e Contextualização do Projeto

Nos Roteiros 1, 2 e 3, trabalhamos com scripts JavaScript concentrados em um único arquivo (`app.js`). Embora essa abordagem seja excelente para aprender os fundamentos, ela se torna inviável em aplicações reais de médio e grande porte, onde a mistura de seletores do DOM, regras de validação e persistência gera o chamado *"código espaguete"*.

No **Roteiro Prático 04**, damos o passo definitivo rumo à arquitetura profissional de software no

frontend. Desenvolveremos a **Vitrine Comunitária de Empreendedores Locais de Garopaba**, uma aplicação que permite cadastrar, listar, filtrar e gerenciar serviços da comunidade local.

Este projeto serve também como a **base tecnológica inicial para a Ação Curricularizada de Extensão na SNCT (Semana Nacional de Ciência e Tecnologia)**.

1.2 Estrutura Arquitetural de Pastas

O projeto deve ser organizado na pasta `roteiro-4/exercicio-roteiro4/` seguindo rigorosamente a estrutura modular:

Árvore de Diretórios Modular do Projeto:

```
roteiro-4/exercicio-roteiro4/
|-- index.html           <-- Estrutura da vitrine e formulários (Bootstrap 5)
|-- styles.css           <-- Estilizações personalizadas e transições
`-- js/
    |-- main.js           <-- Ponto de entrada modular (orquestrador de eventos)
    |-- services/
    |   `-- vitrineService.js <-- Regras de negócio e CRUD no LocalStorage
    |-- views/
    |   `-- vitrineView.js   <-- Geração dinâmica de HTML e Toasts
    `-- utils/
        `-- formatters.js    <-- Funções utilitárias puras (moeda, fone, sanitização)
```

1.3 Fundamentação Teórica: Como Funcionam os ES6 Modules

1.3.1 1. Declaração no HTML (`type="module"`)

Ao declarar `<script type="module" src="js/main.js"></script>`, o navegador aplica comportamentos modernos essenciais:

- **Modo Estrito Automático (`"use strict"`):** Impede o uso de variáveis não declaradas e falhas silenciosas;
- **Escopo Fechado de Módulo:** Variáveis e funções criadas dentro de um arquivo não poluem o escopo global (`window`);
- **Carregamento Assíncrono com Adiamento (`defer`):** O script aguarda o parsing completo do DOM antes de executar.

1.3.2 2. Exportação e Importação (`export` e `import`)

Para compartilhar recursos entre arquivos, usamos a sintaxe explícita de módulos:

Exemplo de Exportação Nomeada e Importação:

```
// No arquivo js/utils/formatters.js
export function formatarTelefone(telefone) {
  return telefone.replace(/^(\d{2})(\d{5})(\d{4})/, '($1) $2-$3');
}

// No arquivo js/main.js
import { formatarTelefone } from './utils/formatters.js';
console.log(formatarTelefone('48999998888')); // (48) 99999-8888
```

1.4 Passo a Passo Guiado do Roteiro

Passo 1: Estruturação dos Utilitários Puros (js/utils/formatters.js)

Crie o arquivo de funções utilitárias que realizam operações puras de transformação de dados:

```
// js/utils/formatters.js
export function formatarMoeda(valor) {
  const num = parseFloat(valor) || 0;
  return num.toLocaleString('pt-BR', { style: 'currency', currency: 'BRL' });
}

export function formatarTelefone(fone) {
  const digits = (fone || '').replace(/\D/g, '');
  if (digits.length === 11) {
    return digits.replace(/^(\d{2})(\d{5})(\d{4})/, '($1) $2-$3');
  }
  return fone;
}

export function escapeHtml(texto) {
  const div = document.createElement('div');
  div.textContent = texto;
  return div.innerHTML;
}
```

Passo 2: Camada de Dados e Serviços (js/services/vitrineService.js)

Esta camada gerencia o ciclo de vida dos dados no localStorage sem qualquer manipulação de elementos HTML:

```
// js/services/vitrineService.js
const STORAGE_KEY = 'garopaba_vitrine_servicos';

export function obterServicos() {
  const dados = localStorage.getItem(STORAGE_KEY);
  return dados ? JSON.parse(dados) : [];
}

export function salvarServico(novoServico) {
  const servicos = obterServicos();
  const servicoCompleto = {
    id: Date.now().toString(),
    dataCadastro: new Date().toISOString(),
    ...novoServico
  };
  servicos.push(servicoCompleto);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
  return servicoCompleto;
}

export function removerServico(id) {
  const servicos = obterServicos();
  const filtrados = servicos.filter(s => s.id !== id);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(filtrados));
  return filtrados;
}
```

Passo 3: Camada de Renderização da Interface (js/views/vitrineView.js)

Esta camada constrói os elementos visuais do Bootstrap 5 e dispara notificações dinâmicas:

```
// js/views/vitrineView.js
import { formatarMoeda, formatarTelefone, escapeHtml } from '../utils/formatters.js';

export function renderizarCards(servicos, containerElement) {
  if (!servicos || servicos.length === 0) {
    containerElement.innerHTML = `
      <div class="col-12 text-center py-5">
        <i class="bi bi-inbox fs-1 text-muted"></i>
        <h5 class="text-muted mt-2">Nenhum serviço cadastrado nesta categoria.</h5>
      </div>`;
    return;
  }

  containerElement.innerHTML = servicos.map(s => `
    <div class="col-md-6 col-lg-4 mb-4">
      <div class="card h-100 shadow-sm border-0 vitrine-card">
        <div class="card-body">
          <div class="d-flex justify-content-between align-items-center mb-2">
            <span class="badge bg-success">${escapeHtml(s.categoria)}</span>
            <span class="fw-bold text-primary">${formatarMoeda(s.precoBase)}</span>
          </div>
          <h5 class="card-title fw-bold mb-1">${escapeHtml(s.nome)}</h5>
          <h6 class="text-muted small mb-3"><i class="bi bi-geo-alt-fill text-danger me-1"></i>
          <p class="card-text text-secondary small">${escapeHtml(s.descricao)}</p>
        </div>
        <div class="card-footer bg-transparent border-0 pt-0 pb-3 d-flex justify-content-between">
          <a href="https://wa.me/55${(s.telefone || '').replace(/\D/g, '')}" target="_blank">
            <i class="bi bi-whatsapp me-1"></i>${formatarTelefone(s.telefone)}
          </a>
          <button class="btn btn-sm btn-outline-danger btn-excluir" data-id="${s.id}" title="Excluir">
            <i class="bi bi-trash"></i>
          </button>
        </div>
      </div>
    </div>
  `).join('');
}

export function exibirToast(mensagem, tipo = 'success') {
  const toastEl = document.getElementById('toastNotificacao');
  const toastBody = document.getElementById('toastMensagem');

  if (toastEl && toastBody) {
    toastBody.innerText = mensagem;
  }
}
```

Passo 4: Orquestrador de Eventos (js/main.js)

Conecte todas as camadas no arquivo central, implementando a validação visual do formulário:

```
// js/main.js
import { obterServicos, salvarServico, removerServico } from './services/vitrineService.js';
import { renderizarCards, exibirToast } from './views/vitrineView.js';

document.addEventListener('DOMContentLoaded', () => {
  const form = document.getElementById('formCadastro');
  const container = document.getElementById('vitrineContainer');
  const filtroCategoria = document.getElementById('filtroCategoria');
  const contadorEl = document.getElementById('totalServicos');

  function atualizarInterface() {
    const todos = obterServicos();
    const filtro = filtroCategoria.value;
    const filtrados = filtro === 'todas' ? todos : todos.filter(s => s.categoria === filtro);
    renderizarCards(filtrados, container);
    if (contadorEl) contadorEl.innerText = `${todos.length} cadastrados`;
  }

  // Validação e Cadastro
  form.addEventListener('submit', (e) => {
    e.preventDefault();
    if (!form.checkValidity()) {
      e.stopPropagation();
      form.classList.add('was-validated');
      return;
    }

    const novo = {
      nome: document.getElementById('nome').value.trim(),
      categoria: document.getElementById('categoria').value,
      bairro: document.getElementById('bairro').value.trim(),
      precoBase: document.getElementById('precoBase').value,
      telefone: document.getElementById('telefone').value.trim(),
      descricao: document.getElementById('descricao').value.trim()
    };

    salvarServico(novo);
    form.reset();
    form.classList.remove('was-validated');
    atualizarInterface();
    exibirToast('Empreendimento cadastrado com sucesso!');
```

1.5 Checklist de Conclusão e Critérios Avaliativos

Antes de realizar o commit final no repositório, certifique-se de validar todos os itens:

- ☐ O projeto utiliza estritamente o padrão ES6 Modules com a tag `<script type="module" src="js/main.js"></script>`;
- ☐ A estrutura em pastas (`js/services/`, `js/views/`, `js/utils/`) foi rigorosamente respeitada;
- ☐ O arquivo `vitrineService.js` opera puramente sobre o `localStorage` sem manipular elementos do DOM;
- ☐ O formulário exibe validações visuais nativas do Bootstrap 5 (`.is-invalid`) quando campos obrigatórios não são preenchidos;
- ☐ A vitrine reage instantaneamente aos filtros de categoria sem recarregar a página;
- ☐ O código está devidamente publicado e em pleno funcionamento no **GitHub Pages**.